AI ASSISTED CODING

Lab Assignment-16.1

Name : T.Shylasri

H.T.NO: 2303A51876

Batch-14(LAB-16)

Date: 30-03-2026

Lab 16 – Database Design and Queries: Schema Design and SQL Generation

Lab Objectives:

• Understand schema design principles for relational databases.

• Use AI-assisted tools to generate SQL schema definitions.

• Write and refine SQL queries (CRUD + aggregate operations).

• Explore how AI can optimize queries and ensure normalization.

Task 1 – Student Information System Schema

Task:

Design a database schema for a Student Information System and generate queries using AI.

Instructions:

• Tables: Students, Courses, Enrollments.

• Define primary keys, foreign keys, and relationships.

• Generate queries:

  o Insert a new student record.

  o Fetch all courses enrolled by a student.

  o Count number of students in each course.

Expected Output:

• SQL CREATE TABLE statements and queries for student – course relationships.

Primary Key

A primary key is a field in a table that uniquely identifies each record. It cannot be null or have duplicate values.

Foreign Key

A foreign key is a field that links one table to another table. It refers to the primary key of another table.

Code

```sql
-- ==========================================
-- CREATE TABLES
-- ==========================================

-- Students Table
CREATE TABLE Students (
    StudentID INTEGER PRIMARY KEY AUTOINCREMENT,
    FirstName TEXT,
    LastName TEXT,
    DateOfBirth DATE
);

-- Courses Table
CREATE TABLE Courses (
    CourseID INTEGER PRIMARY KEY AUTOINCREMENT,
    CourseName TEXT,
    Credits INTEGER
);

-- Enrollments Table (Junction Table)
CREATE TABLE Enrollments (
    EnrollmentID INTEGER PRIMARY KEY AUTOINCREMENT,
    StudentID INTEGER,
    CourseID INTEGER,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);
```

```sql
-- ==========================================
-- INSERT SAMPLE DATA (USER INPUT)
-- ==========================================

-- Insert Students
INSERT INTO Students (FirstName, LastName, DateOfBirth)
VALUES ('John', 'Doe', '2000-01-01');

INSERT INTO Students (FirstName, LastName, DateOfBirth)
VALUES ('Alice', 'Smith', '2001-05-10');

-- Insert Courses
INSERT INTO Courses (CourseName, Credits)
VALUES ('Database Systems', 4);

INSERT INTO Courses (CourseName, Credits)
VALUES ('Operating Systems', 3);

-- Insert Enrollments
INSERT INTO Enrollments (StudentID, CourseID)
VALUES (1, 1);

INSERT INTO Enrollments (StudentID, CourseID)
VALUES (1, 2);

INSERT INTO Enrollments (StudentID, CourseID)
VALUES (2, 1);

-- ==========================================
-- QUERIES
-- ==========================================

-- 1. Insert a New Student Record
INSERT INTO Students (FirstName, LastName, DateOfBirth)
VALUES ('Bob', 'Brown', '2002-03-15');

-- 2. Fetch All Courses Enrolled by a Student (StudentID = 1)
SELECT c.CourseName, c.Credits
FROM Enrollments e
JOIN Courses c ON e.CourseID = c.CourseID
WHERE e.StudentID = 1;

-- 3. Count Number of Students in Each Course
SELECT c.CourseName, COUNT(e.StudentID) AS NumberOfStudents
FROM Courses c
LEFT JOIN Enrollments e ON c.CourseID = e.CourseID
GROUP BY c.CourseID;
Select * from Students;
```

Queries Output

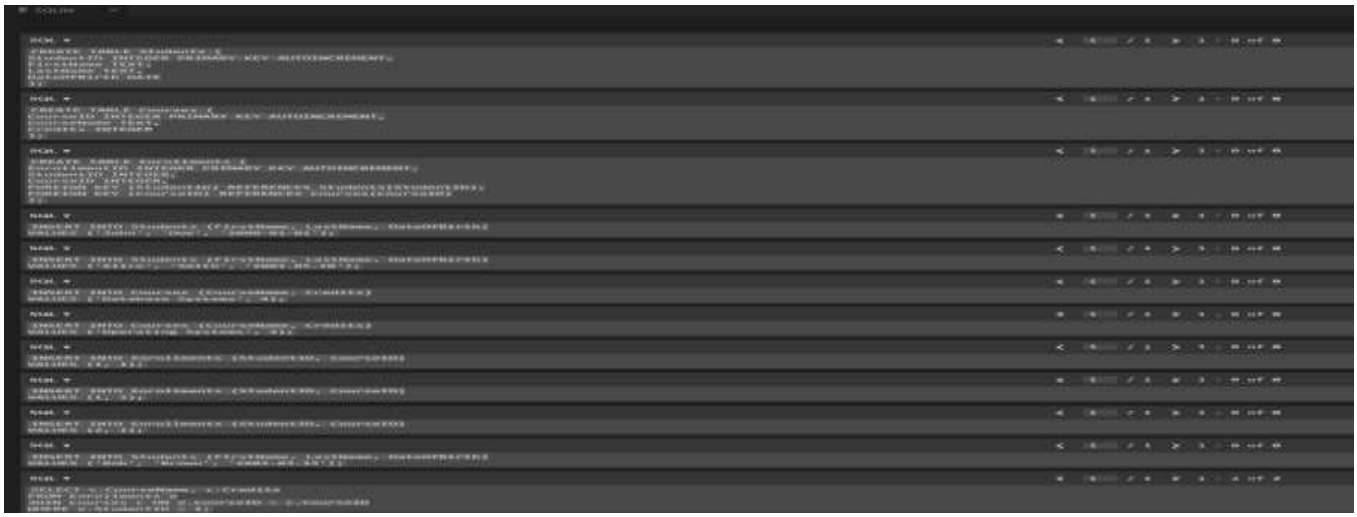


Table Output

SQL ▾ 1 / 1 1 - 2 of 2

CourseName	Credits
Database Systems	4
Operating Systems	3

SQL ▾ 1 / 1 1 - 2 of 2

CourseName	NumberOfStudents
Database Systems	2
Operating Systems	1

SQL ▾ 1 / 1 1 - 3 of 3

StudentID	FirstName	LastName	DateOfBirth
1	John	Doe	2000-01-01
2	Alice	Smith	2001-05-10
3	Bob	Brown	2002-03-15

Observation

- The database contains Students, Courses, and Enrollments tables.
- A many-to-many relationship is implemented using the Enrollments table.
- Queries successfully perform insert, retrieval, and aggregation.

Explanation

- Students and Courses are independent entities.
- Enrollments acts as a junction table connecting them.
- JOIN operations help fetch related data, and COUNT() is used for aggregation.

Justification

- The schema is normalized (no redundancy).
- Primary and foreign keys ensure data integrity.
- Efficient querying is achieved using relationships.

Task 2 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

- Tables: Doctors, Patients, Appointments.
- Use AI to define constraints (unique IDs, valid dates).
- Generate queries:
    - o List all appointments for a specific doctor.
    - o Retrieve patient history by patient ID.
    - o Count total patients treated by each doctor.

Expected Output:

- Normalized schema and SQL queries with joins.

Code

```sql
-- Doctors Table
CREATE TABLE Doctors (
    DoctorID INTEGER PRIMARY KEY AUTOINCREMENT,
    Name TEXT NOT NULL,
    Specialization TEXT,
    UNIQUE(Name, Specialization)
);

-- Patients Table
CREATE TABLE Patients (
    PatientID INTEGER PRIMARY KEY AUTOINCREMENT,
    Name TEXT NOT NULL,
    DateOfBirth DATE NOT NULL
);

-- Appointments Table
CREATE TABLE Appointments (
    AppointmentID INTEGER PRIMARY KEY AUTOINCREMENT,
    DoctorID INTEGER,
    PatientID INTEGER,
    AppointmentDate DATE NOT NULL,
    Diagnosis TEXT,
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID),
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)
);
```

```sql
-- Insert Doctors
INSERT INTO Doctors (Name, Specialization) VALUES ('Dr. Smith', 'Cardiology');
INSERT INTO Doctors (Name, Specialization) VALUES ('Dr. John', 'Neurology');

-- Insert Patients
INSERT INTO Patients (Name, DateOfBirth) VALUES ('Alice', '1995-06-15');
INSERT INTO Patients (Name, DateOfBirth) VALUES ('Bob', '1990-03-28');

-- Insert Appointments
INSERT INTO Appointments (DoctorID, PatientID, AppointmentDate, Diagnosis)
VALUES (1, 1, '2025-03-01', 'Heart Checkup');

INSERT INTO Appointments (DoctorID, PatientID, AppointmentDate, Diagnosis)
VALUES (1, 2, '2025-03-05', 'BP Issue');

INSERT INTO Appointments (DoctorID, PatientID, AppointmentDate, Diagnosis)
VALUES (2, 1, '2025-03-10', 'Migraine');

-- Query 1: List all appointments for a specific doctor
INSERT INTO Appointments (DoctorID, PatientID, AppointmentDate, Diagnosis)
VALUES (2, 1, '2025-03-10', 'Migraine');
SELECT d.Name AS DoctorName, p.Name AS PatientName, a.AppointmentDate, a.Diagnosis
FROM Appointments a
JOIN Doctors d ON a.DoctorID = d.DoctorID
JOIN Patients p ON a.PatientID = p.PatientID
WHERE d.DoctorID = 1;
-- Query 2: Retrieve patient history by patient ID
SELECT p.Name AS PatientName, d.Name AS DoctorName, a.AppointmentDate, a.Diagnosis
FROM Appointments a
JOIN Patients p ON a.PatientID = p.PatientID
JOIN Doctors d ON a.DoctorID = d.DoctorID
WHERE p.PatientID = 1;
-- Query 3: Count total patients treated by each doctor
SELECT d.Name AS DoctorName, COUNT(DISTINCT a.PatientID) AS TotalPatients
FROM Doctors d
LEFT JOIN Appointments a ON d.DoctorID = a.DoctorID
GROUP BY d.DoctorID;
```

Queries Output

Output

DoctorName	PatientName	AppointmentDate	Diagnosis
Dr. Smith	Alice	2025-03-01	Heart Checkup
Dr. Smith	Bob	2025-03-05	BP Issue

SQL ▾ < 1 / 1 > 1 - 3 of 3

PatientName	DoctorName	AppointmentDate	Diagnosis
Alice	Dr. Smith	2025-03-01	Heart Checkup
Alice	Dr. John	2025-03-10	Migraine
Alice	Dr. John	2025-03-10	Migraine

SQL ▾ < 1 / 1 > 1 - 2 of 2

DoctorName	TotalPatients
Dr. Smith	2
Dr. John	1

Observation

- Tables Doctors, Patients, and Appointments are created.
- Relationships are established using foreign keys.
- Queries retrieve appointments and patient history correctly.

Explanation

- Doctors and Patients are linked through Appointments.
- Each appointment stores date and relationship details.
- Queries use JOINs to combine doctor and patient data.

Justification

- Constraints like unique IDs and valid dates improve data accuracy.
- Normalization reduces redundancy.
- Structured design ensures easy data retrieval.

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

- Tables: Books, Members, Loans.
- Use AI to suggest indexing strategy for faster queries.
- Generate queries:
  - o Retrieve all books currently issued.
  - o Find overdue books (loan date > 30 days).
  - o Count number of books loaned by each member.

Expected Output:

- Schema + SQL queries demonstrating joins and conditions.

Code

File Edit Selection View Go Run Terminal Help 1876-AI-LAB

1876-LAB-16.sql

C: > Users > 91986 > OneDrive > Documents > 1876-LAB-16.sql

```sql
-- Books Table
CREATE TABLE Books (
    BookID INTEGER PRIMARY KEY AUTOINCREMENT,
    Title TEXT NOT NULL,
    Author TEXT,
    AvailableCopies INTEGER
);

-- Members Table
CREATE TABLE Members (
    MemberID INTEGER PRIMARY KEY AUTOINCREMENT,
    Name TEXT NOT NULL,
    JoinDate DATE
);

-- Loans Table
CREATE TABLE Loans (
    LoanID INTEGER PRIMARY KEY AUTOINCREMENT,
    BookID INTEGER,
    MemberID INTEGER,
    LoanDate DATE NOT NULL,
    ReturnDate DATE,
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);
-- Indexing Strategy (for faster queries)
CREATE INDEX idx_loans_bookid ON Loans(BookID);
CREATE INDEX idx_loans_memberid ON Loans(MemberID);
CREATE INDEX idx_loans_loandate ON Loans(LoanDate);
```

File Edit Selection View Go Run Terminal Help 1876-AI-LAB-18.4

1876-LAB-16.sql

C: > Users > 91986 > OneDrive > Documents > 1876-LAB-16.sql

```sql
-- Books
INSERT INTO Books (Title, Author, AvailableCopies) VALUES ('DBMS', 'Author A', 5);
INSERT INTO Books (Title, Author, AvailableCopies) VALUES ('OS', 'Author B', 3);

-- Members
INSERT INTO Members (Name, JoinDate) VALUES ('Alice', '2024-01-01');
INSERT INTO Members (Name, JoinDate) VALUES ('Bob', '2024-02-01');

-- Loans
INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate)
VALUES (1, 1, '2025-02-01', NULL); -- currently issued

INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate)
VALUES (2, 2, '2025-01-01', NULL); -- overdue

INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate)
VALUES (1, 2, '2025-03-01', '2025-03-10'); -- returned
-- Query 1: Retrieve all books currently issued
SELECT b.Title, m.Name AS MemberName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL;
-- Query 2: Find overdue books (loan date > 30 days)
SELECT b.Title, m.Name AS MemberName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL
AND DATE(l.LoanDate) <= DATE('now', '-30 days');
-- Query 3: Count number of books loaned by each member
SELECT m.Name AS MemberName, COUNT(l.BookID) AS TotalBooksLoaned
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
GROUP BY m.MemberID;
```

Queries Output

SQLite

```sql
CREATE TABLE Books (
BookID INTEGER PRIMARY KEY AUTOINCREMENT,
Title TEXT NOT NULL,
Author TEXT,
AvailableCopies INTEGER
);
```

```sql
CREATE TABLE Members (
MemberID INTEGER PRIMARY KEY AUTOINCREMENT,
Name TEXT NOT NULL,
JoinDate DATE
);
```

```sql
CREATE TABLE Loans (
LoanID INTEGER PRIMARY KEY AUTOINCREMENT,
BookID INTEGER,
MemberID INTEGER,
LoanDate DATE NOT NULL,
ReturnDate DATE,
FOREIGN KEY (BookID) REFERENCES Books(BookID),
FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);
```

```sql
CREATE INDEX idx_loans_bookid ON Loans(BookID);
```

```sql
CREATE INDEX idx_loans_memberid ON Loans(MemberID);
```

```sql
CREATE INDEX idx_loans_loandate ON Loans(LoanDate);
```

```sql
INSERT INTO Books (Title, Author, AvailableCopies) VALUES ('DBMS', 'Author A', 5);
```

```sql
INSERT INTO Books (Title, Author, AvailableCopies) VALUES ('OS', 'Author B', 3);
```

```sql
INSERT INTO Members (Name, JoinDate) VALUES ('Alice', '2024-01-01');
```

```sql
INSERT INTO Members (Name, JoinDate) VALUES ('Bob', '2024-02-01');
```

```sql
INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate)
VALUES (1, 1, '2025-02-01', NULL);
```

```sql
INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate)
VALUES (2, 2, '2025-01-01', NULL);
```

```sql
INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate)
VALUES (1, 2, '2025-03-01', '2025-03-10');
```

```sql
SELECT b.Title, m.Name AS MemberName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL;
```

Output

Title	MemberName	LoanDate
DBMS	Alice	2025-02-01
OS	Bob	2025-01-01

SQL ▾ 1 / 1 1 - 2 of 2

Title	MemberName	LoanDate
DBMS	Alice	2025-02-01
OS	Bob	2025-01-01

SQL ▾ 1 / 1 1 - 2 of 2

MemberName	TotalBooksLoaned
Alice	1
Bob	2

Observation

- Tables Books, Members, and Loans are used.
- Queries identify issued and overdue books.
- Indexing improves query performance.

Explanation

- Loans table connects Books and Members.
- Conditions (loan date > 30 days) detect overdue books.
- Aggregation counts books borrowed per member.

Justification

- Indexing speeds up search operations.
- Proper relationships maintain consistency.
- Query conditions ensure accurate results.

Task 4 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

• Tables: Users, Products, Orders, OrderDetails.

• Generate queries to:

o Retrieve all orders by a user.

o Find the most purchased product.

o Calculate total revenue in a given month.

• AI should also suggest normalization improvements and query optimization.

Expected Output:

• Complete schema with relationships + SQL queries for analytics.

Code

File Edit Selection View Go Run Terminal Help 1876-AI-LAB-1

1876-LAB-16.sql

C: > Users > 91986 > OneDrive > Documents > 1876-LAB-16.sql

```sql
-- Users Table
CREATE TABLE Users (
    UserID INTEGER PRIMARY KEY AUTOINCREMENT,
    Name TEXT NOT NULL,
    Email TEXT UNIQUE NOT NULL
);

-- Products Table
CREATE TABLE Products (
    ProductID INTEGER PRIMARY KEY AUTOINCREMENT,
    ProductName TEXT NOT NULL,
    Price REAL NOT NULL
);

-- Orders Table
CREATE TABLE Orders (
    OrderID INTEGER PRIMARY KEY AUTOINCREMENT,
    UserID INTEGER,
    OrderDate DATE NOT NULL,
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);

-- OrderDetails Table (Junction Table)
CREATE TABLE OrderDetails (
    OrderDetailID INTEGER PRIMARY KEY AUTOINCREMENT,
    OrderID INTEGER,
    ProductID INTEGER,
    Quantity INTEGER NOT NULL,
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
```

File Edit Selection View Go Run Terminal Help 1876-AI-LAB-18.4

1876-LAB-16.sql

C: > Users > 91986 > OneDrive > Documents > 1876-LAB-16.sql

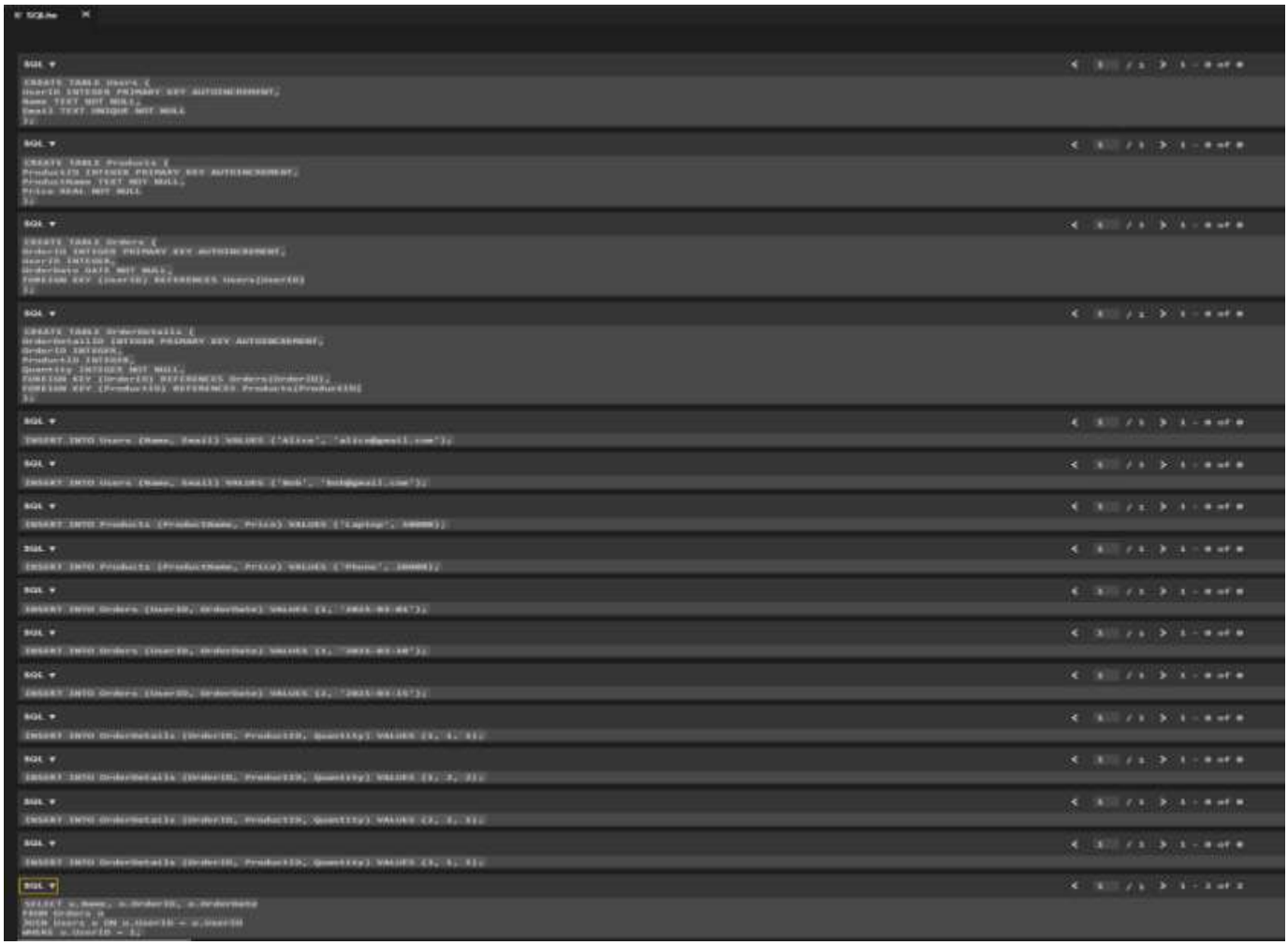
```sql
-- Users
INSERT INTO Users (Name, Email) VALUES ('Alice', 'alice@gmail.com');
INSERT INTO Users (Name, Email) VALUES ('Bob', 'bob@gmail.com');

-- Products
INSERT INTO Products (ProductName, Price) VALUES ('Laptop', 50000);
INSERT INTO Products (ProductName, Price) VALUES ('Phone', 20000);

-- Orders
INSERT INTO Orders (UserID, OrderDate) VALUES (1, '2025-03-01');
INSERT INTO Orders (UserID, OrderDate) VALUES (1, '2025-03-10');
INSERT INTO Orders (UserID, OrderDate) VALUES (2, '2025-03-15');

-- OrderDetails
INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES (1, 1, 1);
INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES (1, 2, 2);
INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES (2, 2, 1);
INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES (3, 1, 1);
-- Query 1: Retrieve all orders by a user
SELECT u.Name, o.OrderID, o.OrderDate
FROM Orders o
JOIN Users u ON o.UserID = u.UserID
WHERE u.UserID = 1;
-- Query 2: Find the most purchased product
SELECT p.ProductName, SUM(od.Quantity) AS TotalSold
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY p.ProductID
ORDER BY TotalSold DESC
LIMIT 1;
-- Query 3: Calculate total revenue in a given month
SELECT SUM(p.Price * od.Quantity) AS TotalRevenue
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
JOIN Orders o ON od.OrderID = o.OrderID
WHERE strftime('%Y-%m', o.OrderDate) = '2025-03';
```

Queries Output



Code

Name	OrderID	OrderDate
Alice	1	2025-03-01
Alice	2	2025-03-10

SQL ▾ 1 / 1 1 - 1 of 1

ProductName	TotalSold
Phone	3

SQL ▾ 1 / 1 1 - 1 of 1

TotalRevenue
160000.0

Observation

- Tables Users, Products, Orders, and OrderDetails are created.
- Queries retrieve user orders, popular products, and revenue.
- Analytical queries are supported.

Explanation

- Orders and OrderDetails handle transactions.
- JOINs connect users with their purchases.
- Aggregate functions calculate revenue and product demand.

Justification

- Normalization avoids duplication of product and user data.
- Optimized queries improve performance.
- Structure supports scalability for large systems.

Task 5 – University Examination Database

Design a database schema for a University Examination System and generate SQL queries using AI.

Instructions:

- Tables: Students, Subjects, Exams, Results.
- Define primary keys, foreign keys, and referential integrity constraints.
- Generate queries:
  - o Insert examination results for a student.
  - o Retrieve subject-wise marks for a student.
  - o Calculate average marks for each subject.

Expected Output:

- Normalized schema and SQL queries involving aggregation and joins.

Code

File Edit Selection View Go Run Terminal Help 1876-AI-LAB-

1876-LAB-16.sql

C: > Users > 91986 > OneDrive > Documents > 1876-LAB-16.sql

```sql
-- Students Table
CREATE TABLE Students (
    StudentID INTEGER PRIMARY KEY AUTOINCREMENT,
    Name TEXT NOT NULL
);

-- Subjects Table
CREATE TABLE Subjects (
    SubjectID INTEGER PRIMARY KEY AUTOINCREMENT,
    SubjectName TEXT NOT NULL
);

-- Exams Table
CREATE TABLE Exams (
    ExamID INTEGER PRIMARY KEY AUTOINCREMENT,
    SubjectID INTEGER,
    ExamDate DATE NOT NULL,
    FOREIGN KEY (SubjectID) REFERENCES Subjects(SubjectID)
);

-- Results Table (Junction Table)
CREATE TABLE Results (
    ResultID INTEGER PRIMARY KEY AUTOINCREMENT,
    StudentID INTEGER,
    ExamID INTEGER,
    Marks INTEGER CHECK (Marks >= 0 AND Marks <= 100),
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (ExamID) REFERENCES Exams(ExamID)
);
```

File Edit Selection View Go Run Terminal Help 1876-AI-LAB

1876-LAB-16.sql

C: > Users > 91986 > OneDrive > Documents > 1876-LAB-16.sql

```sql
-- Students
INSERT INTO Students (Name) VALUES ('Alice');
INSERT INTO Students (Name) VALUES ('Bob');

-- Subjects
INSERT INTO Subjects (SubjectName) VALUES ('Math');
INSERT INTO Subjects (SubjectName) VALUES ('Science');

-- Exams
INSERT INTO Exams (SubjectID, ExamDate) VALUES (1, '2025-03-01');
INSERT INTO Exams (SubjectID, ExamDate) VALUES (2, '2025-03-05');

-- Results
INSERT INTO Results (StudentID, ExamID, Marks) VALUES (1, 1, 85);
INSERT INTO Results (StudentID, ExamID, Marks) VALUES (1, 2, 90);
INSERT INTO Results (StudentID, ExamID, Marks) VALUES (2, 1, 75);
-- Query 1: Insert examination result for a student
INSERT INTO Results (StudentID, ExamID, Marks)
VALUES (2, 2, 88);
-- Query 2: Retrieve subject-wise marks for a student
SELECT s.SubjectName, r.Marks
FROM Results r
JOIN Exams e ON r.ExamID = e.ExamID
JOIN Subjects s ON e.SubjectID = s.SubjectID
WHERE r.StudentID = 1;
-- Query 3: Calculate average marks for each subject
SELECT s.SubjectName, AVG(r.Marks) AS AverageMarks
FROM Results r
JOIN Exams e ON r.ExamID = e.ExamID
JOIN Subjects s ON e.SubjectID = s.SubjectID
GROUP BY s.SubjectID;
Select * from Results;
```

Queries Output

SQLite

```sql
CREATE TABLE Students (
StudentID INTEGER PRIMARY KEY AUTOINCREMENT,
Name TEXT NOT NULL
);
```

```sql
CREATE TABLE Subjects (
SubjectID INTEGER PRIMARY KEY AUTOINCREMENT,
SubjectName TEXT NOT NULL
);
```

```sql
CREATE TABLE Exams (
ExamID INTEGER PRIMARY KEY AUTOINCREMENT,
SubjectID INTEGER,
ExamDate DATE NOT NULL,
FOREIGN KEY (SubjectID) REFERENCES Subjects(SubjectID)
);
```

```sql
CREATE TABLE Results (
ResultID INTEGER PRIMARY KEY AUTOINCREMENT,
StudentID INTEGER,
ExamID INTEGER,
Marks INTEGER CHECK (Marks >= 0 AND Marks <= 100),
FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
FOREIGN KEY (ExamID) REFERENCES Exams(ExamID)
);
```

```sql
INSERT INTO Students (Name) VALUES ('Alice');
```

```sql
INSERT INTO Students (Name) VALUES ('Bob');
```

```sql
INSERT INTO Subjects (SubjectName) VALUES ('Math');
```

```sql
INSERT INTO Subjects (SubjectName) VALUES ('Science');
```

```sql
INSERT INTO Exams (SubjectID, ExamDate) VALUES (1, '2025-03-01');
```

```sql
INSERT INTO Exams (SubjectID, ExamDate) VALUES (2, '2025-03-05');
```

```sql
INSERT INTO Results (StudentID, ExamID, Marks) VALUES (1, 1, 85);
```

```sql
INSERT INTO Results (StudentID, ExamID, Marks) VALUES (1, 2, 90);
```

```sql
INSERT INTO Results (StudentID, ExamID, Marks) VALUES (2, 1, 75);
```

```sql
INSERT INTO Results (StudentID, ExamID, Marks)
VALUES (2, 2, 88);
```

```sql
SELECT s.SubjectName, r.Marks
FROM Results r
JOIN Exams e ON r.ExamID = e.ExamID
JOIN Subjects s ON e.SubjectID = s.SubjectID
WHERE r.StudentID = 1;
```

Output

SubjectName	Marks
Math	85
Science	90

SQL ▾ 1 / 1 1 - 2 of 2

SubjectName	AverageMarks
Math	80.0
Science	89.0

SQL ▾ 1 / 1 1 - 4 of 4

ResultID	StudentID	ExamID	Marks
1	1	1	85
2	1	2	90
3	2	1	75
4	2	2	88

Observation

- Tables Students, Subjects, Exams, and Results are used.
- Queries insert results and calculate averages.
- Relationships ensure correct mapping of marks.

Explanation

- Results table links students with subjects and exams.
- Aggregation functions compute average marks.
- JOINs retrieve subject-wise results.

Justification

- Referential integrity ensures valid data.
- Normalization avoids redundancy.
- Queries support academic evaluation.